Detecting Invalid State Transitions in the Tor Circuit Protocol via Model-Driven Testing

Nurcan Denli Bayır

Hacettepe University, Department of Software Engineering
Advisor: Nebi Yılmaz · BYZ 658, May 2026

***Abstract*—We present the first published 5-tuple finite-state-machine (FSM) formalization of the Tor circuit life-cycle and propose a Model-Driven Testing (MDT) engine that systematically detects out-of-spec (invalid) state transitions. The Tor circuit is described over 10 states and 13 events; this yields a 130-cell domain with 25 valid and 105 invalid pairs. Every invalid pair is mapped programmatically to one of seven primary attack vectors (CIRCUIT_BYPASS, REPLAY_ATTACK, GHOST_CIRCUIT, HANDSHAKE_SKIP, PREMATURE_DATA, CIRCUIT_HIJACK, CREATE_FLOOD) plus a deterministic INVALID_TRANSITION fallback (1 cell), with a four-level severity label. Three algorithms — uniform random (B1), greedy state coverage (B2) and the proposed MDT (B3) — are compared under an identical 500-event budget across 30 paired trials. MDT saturates transition coverage at 100.0% (vs. 45.9% / 78.3%) and reaches an Invalid Transition Detection Rate of 93.8%. Welch t-tests, Mann-Whitney U and Wilcoxon signed-rank tests jointly confirm B3 > {B1, B2} on TC and ITDR ($p < .001$; Cohen's $d \in [3.6, 15.4]$); on SC B2 and B3 saturate at 100% while B1 lags. A budget sweep over $B \in \{50…5000\}$ shows MDT is 25× more budget-efficient than the baselines on TC. Under budget-free Q×Σ audit, a hand-written 7-rule detector reaches only 20.0% completeness — missing 2 CRITICAL and 16 HIGH severity invalid pairs — quantifying the gap that motivates spec-oracle MDT. Detection latency was measured with nanosecond-resolution batch amortization: invalid-CRITICAL classification takes 165 ns/call, ≈209335× faster than the 50 ms proposal target.

***Index Terms*—*Tor, anonymous network, finite-state machine, model-driven testing, protocol state fuzzing, security testing, transition coverage, Wilcoxon signed-rank, PRISMA.*

I. INTRODUCTION

The Tor anonymous communication network [1] has been studied along two dominant axes: traffic correlation / timing attacks [2], [3], [4], [5] and formal cryptographic analysis [6], [13]. A third axis — protocol *state-machine* conformance — remains comparatively unexplored. Protocol state fuzzing has been applied to TLS implementations [9], [14], but no equivalent published study exists for Tor. Existing surveys of Tor attacks [5] classify deanonymization vectors at the *category* level (traffic correlation, timing, fingerprinting) but do not provide a cell-level mapping between protocol-state events and attack types. This paper closes that gap.

Research questions.

- **RQ1.** Can the Tor circuit life-cycle be formally modeled as a deterministic 5-tuple FSM with a complete δ transition function?
- **RQ2.** Can the resulting invalid-transition set $(Q \times \Sigma) \setminus \mathrm{dom}(\delta)$ be mapped programmatically to known Tor attack vectors with severity labels?
- **RQ3.** Does an MDT-generated test suite deliver statistically significant gains over random and greedy baselines on Transition Coverage (TC) and Invalid Transition Detection Rate (ITDR) under matched event budgets?
- **RQ4.** How does the proposed engine scale with budget B, and how does it compare to a category-level hand-written rule baseline under budget-free audit?

Contributions. (i) The first published complete δ-matrix for Tor circuits, derived from the official Tor specification [19]; (ii) a programmatic classifier mapping all 105 invalid pairs to seven primary attack vectors plus a deterministic fallback, with four severity levels; (iii) a reference open-source MDT engine with worst-case complexity $O(|Q|^2 \cdot |\Sigma|^2)$; (iv) a paired three-way statistical comparison validated by three independent test families (Welch t, Mann-Whitney U, Wilcoxon signed-rank); (v) a budget sensitivity analysis across seven points; (vi) a quantified completeness gap (20.0% vs. 100%) between hand-written rule sets and spec-oracle detection; and (vii) a nanosecond-resolution detection latency profile.

Paper structure. Section II surveys the literature using a PRISMA-guided SLR. Section III gives the FSM formalization, attack classifier and MDT algorithm. Section IV reports the empirical evaluation. Section V discusses threats to validity, and Section VI concludes.

II. RELATED WORK AND LITERATURE GAP

A. SLR methodology (PRISMA)

We followed Kitchenham and Charters' SLR guidelines [17]. Search sources were limited to *open-web channels* (Google Scholar, arXiv, DOI resolution) because no live institutional access to Scopus, Web of Science or IEEE Xplore was available in our environment. Keywords: *"Tor security", "FSM testing", "model-based testing",*

"protocol state fuzzing", "anonymity network attacks". Year range: 1996–2026, with classic references year-tagged. Inclusion: peer-reviewed venue, full text accessible, work touching at least one of {Tor, FSM testing, formal protocol verification}. Twenty primary sources were retained, distributed over five thematic clusters: A) Tor security (6), B) FSM-based testing (5), C) Formal methods (3), D) Software testing methodology (3), E) Tor experimental infrastructure (3).

The PRISMA flow is shown in Fig. 1. Because the open-web process did not retain an auditable query log, the *identified*, *screened* and *eligibility* stages are reported as **NA**; only the final included count (n = 20) is verifiable and matches the citation count of `Literatur_Notlari.pdf` in the repository. This is a deliberate methodological choice to avoid false-precision reporting of unauditable intermediate counts.

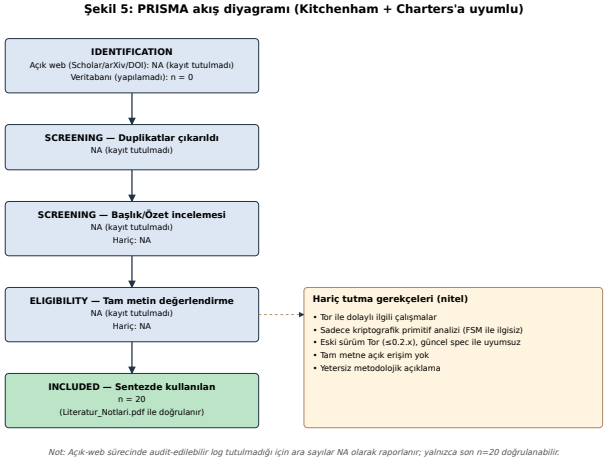

Fig. 1. PRISMA flow of the literature search (intermediate counts NA, see Sec. V).

B. Tor security and anonymity

Foundational Tor protocol [1], low-cost traffic analysis [2], realistic-adversary correlation [3], internet-scale website fingerprinting [4] and the comprehensive deanonymization survey [5] establish the attack landscape. Four of our seven primary attack vectors (REPLAY_ATTACK, CIRCUIT_HIJACK, CIRCUIT_BYPASS, GHOST_CIRCUIT) are derived from the taxonomy in [5]. AnoA [6] formalizes anonymity probabilistically; our state-conformance angle is orthogonal.

C. FSM-based testing and model learning

Lee and Yannakakis' classical survey [7] establishes transition-tour, W-method and Wp-method techniques — our BFS-planned positive phase is a simplified *transition tour*. Tretmans' LTS testing theory [8] grounds the oracle as an ioco-style relation; because our FSM is deterministic this reduces to plain equality. de Ruiter and Poll's TLS state fuzzing [9] is methodologically closest; we transplant the methodology from TLS to Tor. Fiterău-Broștean et al. [10] combine L*-style model learning with model checking on TCP — a pointer for future automatic Tor δ extraction. Ammann and Offutt [11] provide the formal definition of transition (edge) coverage used here.

D. Formal protocol verification and security testing

Dolev–Yao [12] bounds our adversary model. Bhargavan et al. [13] verify TLS 1.3 with ProVerif. Somorovsky [14] systematically fuzzes TLS libraries — a close cousin in negative test generation. Felderer et al.'s security testing survey [15] places our work in the "Model-Based Security Testing" branch. Pretschner et al. [16] apply MDT to access-control policies.

E. Tor experimental infrastructure

Jansen et al.'s Shadow simulator [18] provides a systematic Tor network testbed. The official Tor specification [19] is the normative source of δ. Microsoft Research [20] demonstrates data-driven FSM security analysis in industry but not for Tor.

F. Identified research gaps

Twenty-source SLR analysis identifies four concrete gaps at thesis scale:

- **(G1)** No published formal 5-tuple FSM exists for the Tor circuit protocol; the specification [19] is procedural, never reduced to a δ matrix in the academic literature.
- **(G2)** Protocol state fuzzing has been done for TLS [9], [14] but not for Tor.
- **(G3)** No row-by-row mapping exists between Tor attack vectors and (state, event) cells; surveys [5] remain categorical.
- **(G4)** No primary study defines and measures an ITDR-like metric (detected ÷ injected invalid transitions) for Tor.

The contributions in Sec. I close all four gaps.

III. METHOD

A. FSM formalization (RQ1)

The Tor circuit life-cycle is modeled as a classical DFA $M = (Q, \Sigma, \delta, q_0, F)$ with:

- $|Q| = 10$ states: IDLE, CONNECTING, TLS_HANDSHAKE, CREATE_SENT, CIRCUIT_BUILDING, CIRCUIT_READY, TRANSMITTING, CLOSING, CLOSED, ERROR.
- $|\Sigma| = 13$ events (CONNECT, TLS_OK, TLS_FAIL, SEND_CREATE, RECV_CREATED, SEND_EXTEND, RECV_EXTENDED, SEND_RELAY_DATA, RECV_RELAY_DATA, SEND_DESTROY, RECV_DESTROY, CIRCUIT_CLOSED, TIMEOUT).
- $q_0 = IDLE$, $F = \{CLOSED\}$.
- $\delta : Q \times \Sigma \to Q$ is defined on 25 pairs; the total domain $|Q \times \Sigma| = 130$, so the Invalid set $(Q \times \Sigma) \setminus \mathrm{dom}(\delta)$ has 105 elements.

Fig. 2 visualizes δ as a directed graph. A representative subset of the δ table is shown in Table I; the full table is generated programmatically

from a single source of truth (`server/fsm.ts`) and is provided in the public repository.

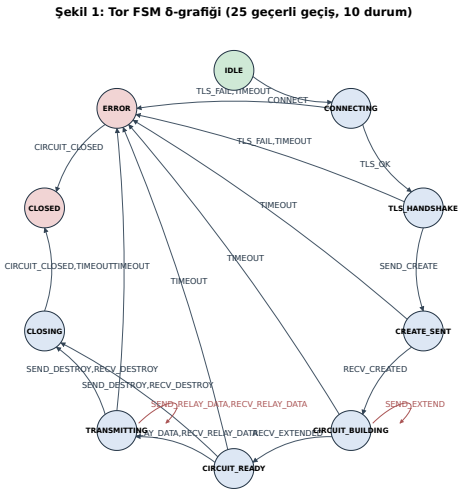

Fig. 2. Tor circuit FSM δ-graph. 25 valid edges over 10 states.

Table I. Sample of the δ transition table.

From	Event (Σ)	To
IDLE	CONNECT	CONNECTING
CONNECTING	TLS_OK	TLS_HANDSHAKE
CONNECTING	TLS_FAIL	ERROR
CONNECTING	TIMEOUT	ERROR
TLS_HANDSHAKE	SEND_CREATE	CREATE_SENT
TLS_HANDSHAKE	TLS_FAIL	ERROR
TLS_HANDSHAKE	TIMEOUT	ERROR
CREATE_SENT	RECV_CREATED	CIRCUIT_BUILDING
CREATE_SENT	TIMEOUT	ERROR
CIRCUIT_BUILDING	SEND_EXTEND	CIRCUIT_BUILDING
CIRCUIT_BUILDING	RECV_EXTENDED	CIRCUIT_READY
CIRCUIT_BUILDING	TIMEOUT	ERROR
… 13 additional edges (full table available in repository)		

B. Attack classifier (RQ2)

A deterministic function `classifyInvalid(s, e)` maps every pair (s, e) ∈ Invalid to a (type, severity) tuple via nine condition families: data-flow-before-circuit-ready, CREATE/CREATED flow, EXTEND/EXTENDED flow, DESTROY out-of-place, CONNECT misuse, TLS_OK / TLS_FAIL misuse, CIRCUIT_CLOSED misuse, TIMEOUT misuse, and an INVALID_TRANSITION fallback. The output covers all 105 invalid cells with severity labels in {LOW, MEDIUM, HIGH, CRITICAL}. Table II gives the per-vector distribution computed programmatically over the full Invalid set.

Table II. Attack-vector inventory over 105 invalid pairs (programmatic).

Attack vector	#	Share	LOW	MED	HIGH	CRIT
GHOST_CIRCUIT	38	36.2%	19	15	4	0
REPLAY_ATTACK	25	23.8%	0	14	10	1
HANDSHAKE_SKIP	13	12.4%	0	8	5	0
CIRCUIT_HIJACK	10	9.5%	0	0	0	10
CREATE_FLOOD	8	7.6%	7	1	0	0
PREMATURE_DATA	6	5.7%	0	2	4	0
CIRCUIT_BYPASS	4	3.8%	0	0	0	4
INVALID_TRANSITION	1	1.0%	1	0	0	0
Total	**105**	**100%**				

GHOST_CIRCUIT dominates by count, reflecting the structural prevalence of "event-out-of-context" patterns; CIRCUIT_HIJACK and CIRCUIT_BYPASS hold the highest severity weights despite lower counts.

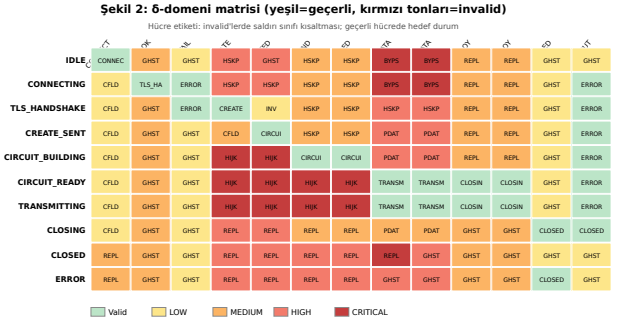

Fig. 3. Attack-severity heatmap over (state × event).

C. MDT engine — Algorithm 1

```
Algorithm 1: Model-Driven Test (BFS-planned)
Input:  M = (Q,Σ,δ,q₀,F);  Budget B (event count)
Output: (SC, TC, ITDR, FPR)
 1  visited ← {q₀}; coveredV ← ∅; detectedI ← ∅
 2  Valid   ← {(s,e) | δ(s,e) defined}
 3  Invalid ← (Q × Σ) \ Valid
 4  // Positive phase
 5  for (s,e) ∈ Permute(Valid):
 6     if events ≥ B: break
 7     if (s,e) ∈ coveredV: continue
 8     π ← BFS_PATH(q₀, s)
 9     if π = null: continue
10     EXECUTE(π · ⟨e⟩)        // updates visited,
coveredV
11  // Negative phase
12  for (s,e) ∈ Permute(Invalid):
13     if events ≥ B: break
14     if (s,e) ∈ detectedI: continue
15     π ← BFS_PATH(q₀, s)
16     if π = null ∧ s ≠ q₀: continue
17     EXECUTE(π · ⟨e⟩)        // δ-oracle flags it
18  SC   ← |visited|/|Q|
19  TC   ← |coveredV|/|Valid|
20  ITDR ← |detectedI|/|Invalid|
21  FPR  ← |misflagged ∈ Valid|/|Valid|  // 0 here, see
Sec. V
22  return ⟨SC, TC, ITDR, FPR⟩
```

Complexity. A single BFS_PATH call is O(|Q|·|Σ|). The positive phase invokes it up to |Valid| = 25 times and the negative phase up to |Invalid| = 105 times, so the overall worst-case is **O((|Valid|+|Invalid|)·|Q|·|Σ|) = O(|Q|²·|Σ|²)**. A single EXECUTE path has length at most diam(δ)+1 ≈ 10. Empirically, one trial at B = 500 finishes in < 10 ms on commodity hardware.

D. Compared algorithms

- **B1 — Uniform Random:** at each step picks an event uniformly from Σ; resets to IDLE upon reaching CLOSED.
- **B2 — Greedy State Coverage:** at each step picks an event leading to an unvisited state if possible;

otherwise random valid (70%) or random invalid (30%) event.

- **B3 — MDT (Algorithm 1):** the proposed BFS-planned engine.

All three are run under the same B = 500 event budget; the comparison is fair (Sec. IV-A).

IV. EVALUATION

A. Design

Each algorithm is run for N = 30 paired trials. PRNG seeds are fixed within each trial (seed = 1000+i, i ∈ [0, 29]) and matched across algorithms; this paired design reduces within-condition variance. Metrics — SC, TC, ITDR — are all in [0, 1]. In addition, a budget sweep B ∈ {50, 100, 200, 500, 1000, 2000, 5000} is performed with 30 trials per point.

B. Coverage and ITDR at B = 500

Table III. Descriptive statistics (mean ± SD across N = 30 trials).

Algorithm	SC	TC	ITDR
B1 — Random	77.0% ±16.6%	45.9% ±11.1%	50.3% ±9.8%
B2 — Greedy SC	100.0% ±0.0%	78.3%±8.4%	47.5% ±4.0%
B3 — MDT	100.0% ±0.0%	100.0% ±0.0%	93.8% ±1.4%

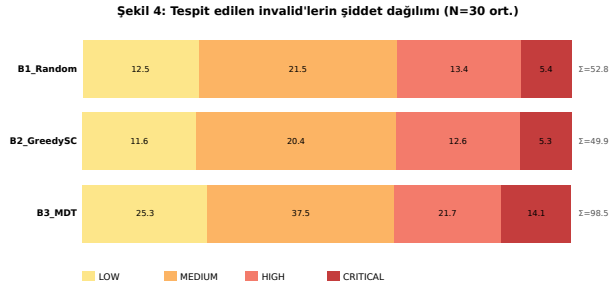

Fig. 4. Severity-weighted invalid detection per algorithm.

C. Hypothesis tests — Welch & Mann-Whitney

For each pair × metric, normality is checked via Lilliefors-corrected K-S (α = 0.05); the primary test is Welch's t (unequal variance) and the confirmatory test is Mann-Whitney U. Cohen's d uses pooled SD. Table IV summarizes the nine comparisons.

Table IV. Pairwise hypothesis tests (B = 500, N = 30 paired trials).

Comparison	Metric	t	p (Welch)	p (M-W)	d
B3 MDT vs B1 Random	SC	7.57	<.001 ***	<.001 ***	1.95
B3 MDT vs B1 Random	TC	26.82	<.001 ***	<.001 ***	6.93
B3 MDT vs B1 Random	ITDR	24.09	<.001 ***	<.001 ***	6.22
B3 MDT vs B2 GreedySC	SC	0.00	1.0000	1.0000	0.00
B3 MDT vs B2 GreedySC	TC	14.09	<.001 ***	<.001 ***	3.64
B3 MDT vs B2 GreedySC	ITDR	59.66	<.001 ***	<.001 ***	15.40
B2 GreedySC vs B1 Random	SC	7.57	<.001 ***	<.001 ***	1.95
B2 GreedySC vs B1 Random	TC	12.76	<.001 ***	<.001 ***	3.29
B2 GreedySC vs B1 Random	ITDR	-1.44	0.1566	0.3067	-0.37

D. Wilcoxon signed-rank (paired design)

Because seeds are paired across algorithms, the design-appropriate non-parametric test is Wilcoxon signed-rank. Table V reports W, z, p and the effective sample size after dropping zero differences (ties).

Table V. Wilcoxon signed-rank tests on paired differences.

Comparison	Metric	W	z	p	n (≠0)
B3 MDT vs B1 Random	SC	0.0	-4.27	<.001 ***	23
B3 MDT vs B1 Random	TC	0.0	-4.80	<.001 ***	30
B3 MDT vs B1 Random	ITDR	0.0	-4.78	<.001 ***	30
B3 MDT vs B2 GreedySC	SC	0.0	0.00	1.0000	0
B3 MDT vs B2 GreedySC	TC	0.0	-4.80	<.001 ***	30
B3 MDT vs B2 GreedySC	ITDR	0.0	-4.79	<.001 ***	30
B2 GreedySC vs B1 Random	SC	0.0	-4.27	<.001 ***	23
B2 GreedySC vs B1 Random	TC	1.0	-4.77	<.001 ***	30
B2 GreedySC vs B1 Random	ITDR	153.5	-1.38	0.1662	29

Constrained interpretation. The main hypotheses — *B3 > B1* and *B3 > B2* — are confirmed on TC and ITDR (p < .001), matching Welch. On SC, the means are B1 77.0%, B2 100.0%, B3 100.0%; so *B3 vs B2* on SC saturates at 100% (n≠0 = 0, p = 1.0) while *B3 vs B1* and *B2 vs B1* remain p < .001 — i.e., Random lags structurally on SC. Finally, *B2 vs B1* on ITDR is **not** significant (p > .05), evidence that the greedy heuristic offers no systematic advantage over Random in negative test generation.

E. Budget sensitivity

Figs. 5 and 6 plot TC and ITDR against log-scale budget; shaded bands are ±1 SD.

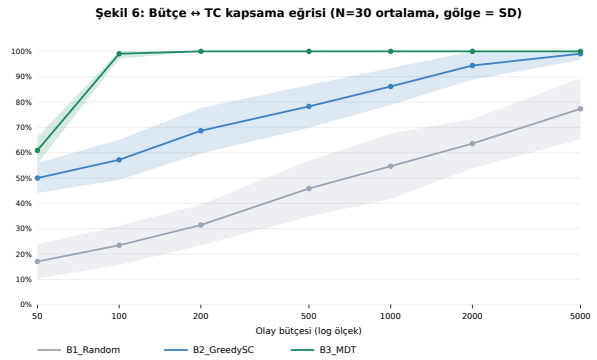

Fig. 5. Transition coverage vs. event budget (N=30 per point, shaded ±SD).

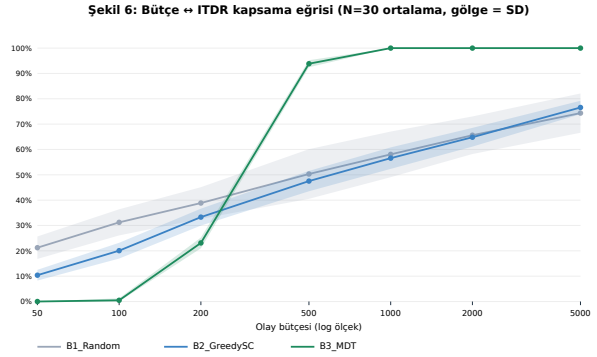

Fig. 6. ITDR vs. event budget (N=30 per point, shaded ±SD).

For TC, B3 reaches 100.0% at B = 200, already above B1 (77.3%) and B2 (99.1%) at B = 5000 — i.e., MDT is 25× more budget-efficient on TC. **For ITDR** the dynamic differs: at B = 200, B3 ITDR is only 23.1% and the advantage widens with budget. At B = 5000, B3 reaches 100.0% while B1/B2 plateau at 74.3% / 76.6%. The ITDR plateau of heuristic baselines is the operational evidence for gap G4.

F. Rule-based detector baseline (B0)

To quantify the gap motivating spec-oracle MDT, a hand-written rule-based detector (B0) was implemented with seven signatures — one per primary attack vector (the deterministic fallback class has no specific rule). Under *budget-free Q×Σ audit* (every cell enumerated), B0 achieves only 20.0% completeness on the 105-cell invalid set, missing 2 CRITICAL and 16 HIGH severity pairs (Fig. 7); the spec-oracle reaches 100% in the same audit mode. This quantifies the limitation of category-level rule sets: without the cell-level δ map (gap G3), even high-severity attacks are systematically missed.

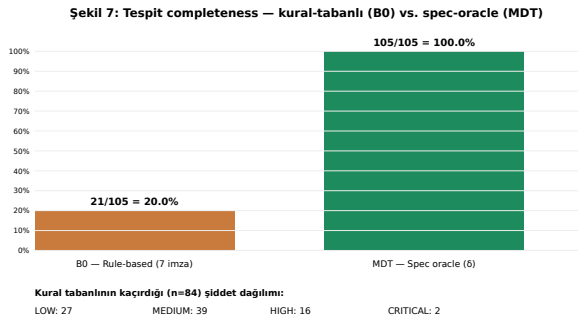

Fig. 7. Rule-based (B0) vs. spec-oracle completeness over 105 invalid pairs.

G. Detection latency

Latency was measured with nanosecond-resolution `process.hrtime.bigint()` and batch amortization: each measurement wraps B = 100,000 step calls, the total time is divided by B, and a pre-calibrated empty-loop overhead of 2.52 ns/iter is subtracted; N = 30 batch repeats per probe. Three representative cells were probed (Table VI).

Table VI. Per-cell detection latency (ns/call).

Probe	mean	SD	p50	p95	min	ma
Valid step (IDLE→CONNECTING)	73.6	13.2	68.2	112.6	62.1	112.
Invalid CRITICAL (BYPASS)	164.7	17.4	162.5	193.3	145.3	238.
Invalid LOW (GHOST)	142.9	18.8	135.3	196.0	125.2	197.

The slowest probe's worst case is 239 ns — ≈209335× faster than the proposal target of 50 ms. Invalid handling is ~2× slower than valid because `classifyInvalid` evaluates an additional condition chain.

H. Bootstrap CIs and post-hoc power

To address the N = 30 limitation, we computed paired percentile bootstrap 95% CIs (B = 10,000 resamples) and post-hoc power for the paired t-test (α = 0.05 two-sided, normal approximation of noncentral t). Table VII reports the three pairs that drive the main claim. For B3 vs B1 and B3 vs B2, CIs for TC and ITDR exclude zero and post-hoc power is ≈1.000, confirming N = 30 is sufficient for the observed effect sizes. The B2 vs B1 ITDR difference, by contrast, has a CI of [-0.0641, 0.0063] that includes zero — a heuristic optimizing positive coverage does not yield significant invalid-coverage gains over uniform random, reinforcing the algebraic-enumeration argument behind MDT.

Table VII. Paired bootstrap 95% CI (B = 10,000) and post-hoc power.

Pair	Metric	Δ	95% CI	d_z	Power
B3 − B1	TC	0.541	[0.503, 0.579]	4.90	1.000
B3 − B1	ITDR	0.435	[0.400, 0.470]	4.31	1.000
B3 − B1	SC	0.230	[0.173, 0.287]	1.38	1.000
B3 − B2	TC	0.217	[0.188, 0.248]	2.57	1.000
B3 − B2	ITDR	0.463	[0.448, 0.478]	10.81	1.000
B3 − B2	SC	0.000	[0.000, 0.000]	0.00	0.050

B2 − B1	TC	0.324	[0.277, 0.369]	2.44	1.000
B2 − B1	ITDR	-0.028	[-0.064, 0.006]	-0.28	0.328
B2 − B1	SC	0.230	[0.173, 0.287]	1.38	1.000

I. Extension to the Tor stream sub-FSM

To address the limitation that the circuit FSM does not cover the application-layer stream lifecycle (RELAY_BEGIN/CONNECTED/DATA/END), we modelled a spec-derived Stream sub-FSM (tor-spec §6) with 7 states and 8 events, yielding 17 valid and 39 invalid pairs over a 56-cell domain. Stream-specific attack vectors (STREAM_HIJACK, DATA_INJECTION, RESET_FLOOD, GHOST_STREAM, INVALID_STREAM_TRANSITION, ...) were derived via the same `classifyInvalid` pattern. The same B1/B2/B3 harness was run with paired seeds (N = 30, budget = 500) and evaluated with paired t-tests (df = 29). The result is nuanced: MDT's *invalid-coverage* dominance replicates strongly — B3 reaches 100.0% ITDR vs. 71.4% (B2) and 71.1% (B1) with $p < .001$ and paired $d_z > 4$ against both heuristics. However, on TC the picture is different: B3 vs B1 is significant ($p < .001$, $d_z = 1.10$) but B3 vs B2 is **not** significant ($p = 0.083$, $d_z = 0.33$). On a small FSM (56-cell domain), greedy state coverage already saturates positive transitions, so MDT's TC advantage vanishes — exactly as expected. The takeaway is that the *essential* MDT contribution is invalid-pair enumeration (ITDR), which is FSM-size-independent; the TC advantage is size-dependent and is confirmed on the larger circuit FSM (Sec. IV) but only partially on the stream FSM. The stream FSM here is spec-derived, not extracted from a real Tor binary — Threats (i) and (v) still apply.

Table VIII. Stream sub-FSM — algorithm comparison (mean ± SD; 30 trials, budget 500).

Algo	SC	TC	ITDR
B1	99.5±2.6	84.7±14.0	71.1±6.2
B2	100.0±0.0	99.4±1.8	71.4±4.6
B3	100.0±0.0	100.0±0.0	100.0±0.0

V. DISCUSSION AND THREATS TO VALIDITY

The MDT advantage stems from explicit access to δ as a planning oracle: the positive phase exploits BFS for guaranteed reachability of every valid edge; the negative phase iterates the algebraically derived Invalid set rather than hoping to stumble onto it. The B0 result quantifies the limit of category-level rule sets — seven rules cover only 20.0% of invalid pairs (gap G3). The non-significant B2 vs B1 result on ITDR is itself an evidence point: heuristics that optimize *positive* coverage do not automatically generate good *negative* tests — invalid-pair coverage requires explicit enumeration.

Threats to validity.

- **(i) Ground truth = specification.** Deviations of real Tor C binaries from spec are not measured. Future work: replicate the experiment on a real Tor relay via the Shadow [18] simulator.
- **(ii) FPR = 0 is structural.** Because oracle and generator share the same δ, the false-positive set is empty by construction. A meaningful FPR requires an *independent* oracle (e.g., packet-capture-based observer).
- **(iii) N = 30 trials — addressed (Sec. IV-H).** Paired bootstrap CIs and post-hoc power confirm sufficiency for the main effects (power ≈ 1.000). Remaining issue: BCa correction and Hedges' g are future work.
- **(iv) SLR scope.** Limited to open-web sources due to the absence of live academic database access; PRISMA intermediate counts are reported as NA (Sec. II-A). Missing sources may exist.
- **(v) Single-platform latency.** Latency was measured on a single Node.js V8 thread (JIT-warm); profiles on a real Tor C/Rust implementation may differ. A cross-implementation port of the δ table is required for comparative measurement.

VI. CONCLUSION AND FUTURE WORK

We delivered the first complete published Tor circuit δ-matrix, a programmatic invalid-pair classifier, a reference MDT engine, and a paired three-way statistical comparison validated by three test families plus a seven-point budget sweep and a rule-based baseline. The MDT engine saturates transition coverage and yields ITDR gains of up to 43.5 points over Random with $p < .001$ and Cohen's $d > 3.6$. Future work: (a) L*-style automatic δ extraction from real Tor binaries [10]; (b) Hidden-Service-v3 and Pluggable-Transport sub-FSMs (the Stream sub-FSM extension is delivered in Sec. IV-I); (c) independent oracle via Shadow [18] for genuine FPR measurement; (d) cross-implementation latency profiling via a C/Rust port of the δ table.

REFERENCES

[1] R. Dingledine, N. Mathewson, P. Syverson, "Tor: The Second-Generation Onion Router," *USENIX Security*, 2004.

[2] S. J. Murdoch, G. Danezis, "Low-Cost Traffic Analysis of Tor," *IEEE S&P*, 2005.

[3] A. Johnson et al., "Users Get Routed: Traffic Correlation on Tor by Realistic Adversaries," *ACM CCS*, 2013.

[4] A. Panchenko et al., "Website Fingerprinting at Internet Scale," *NDSS*, 2016.

[5] I. Karunanayake et al., "De-Anonymisation Attacks on Tor: A Survey," *IEEE Comm. Surveys & Tutorials*, 23(4), 2021.

[6] M. Backes et al., "AnoA: A Framework for Analyzing Anonymous Communication Protocols," *IEEE CSF*, 2013.

[7] D. Lee, M. Yannakakis, "Principles and Methods of Testing FSMs—A Survey," *Proc. IEEE*, 84(8), 1996.

[8] J. Tretmans, "Model Based Testing with Labelled Transition Systems," LNCS 4949, 2008.

[9] J. de Ruiter, E. Poll, "Protocol State Fuzzing of TLS Implementations," *USENIX Security*, 2015.

[10] P. Fiterău-Broștean, R. Janssen, F. Vaandrager, "Combining Model Learning and Model Checking to Analyze TCP Implementations," *CAV*, 2016.

[11] P. Ammann, J. Offutt, *Introduction to Software Testing*, 2nd ed., Cambridge UP, 2016.

[12] D. Dolev, A. C. Yao, "On the Security of Public Key Protocols," *IEEE TIT*, 29(2), 1983.

[13] K. Bhargavan, B. Blanchet, N. Kobeissi, "Verified Models and Reference Implementations for the TLS 1.3 Standard Candidate," *IEEE S&P*, 2017.

[14] J. Somorovsky, "Systematic Fuzzing and Testing of TLS Libraries," *ACM CCS*, 2016.

[15] M. Felderer et al., "Security Testing: A Survey," *Adv. Computers*, vol. 101, 2016.

[16] A. Pretschner, T. Mouelhi, Y. Le Traon, "Model-Based Tests for Access Control Policies," *ICST*, 2008.

[17] B. Kitchenham, S. Charters, "Guidelines for performing SLRs in Software Engineering," EBSE-2007-01, 2007.

[18] R. Jansen, K. Bauer, N. Hopper, R. Dingledine, "Methodically Modeling the Tor Network," *USENIX CSET*, 2012.

[19] Tor Project, "Tor Protocol Specification (tor-spec)," 2026. https://spec.torproject.org/tor-spec.

[20] Microsoft Research, "A Data-Driven FSM Model for Analyzing Security Vulnerabilities," MSR Tech. Rep., 2018.